Task-1: Test Strategy Document (Chain-of-Thought Approach)

🔹 **Prompt**

Context:
Act as a SaaS Domain Testing Expert for the *Prevsis Application*

Instructions:

- Build the test strategy using a structured, step-by-step approach
- Clearly define scope before designing test cases
- Focus on real-world SaaS risks and challenges
- Maintain a strictly professional tone

⚠️ **Important Note**

Testing should not begin without clearly defining:

- Modules
- Workflows
- Risk areas

👉 Without this foundation, the test strategy becomes **theoretical and ineffective for real-world implementation**

🧠 **Step 1: Application Understanding**

Application Type:

- Web Application
- SaaS Platform
- Multi-tenant architecture

Domain:

- Configurable workflow system (Forms → Planning → Execution → Validation)

🧠 **Step 2: Module Identification**

Core Modules:

1. Login
2. Dashboard / MiPrevsis

3. Master Forms
4. Organizational Structure
5. Master Services
6. Planner
7. View Module
8. User Profiling & Management

🧠 **Step 3: Risk Identification**

🔴 **High-Risk Areas**

- Role-Based Access Control (RBAC: Root/Admin/Basic/Medium)
- Dynamic form creation (data integrity and validation risks)
- Multi-tenant data isolation
- Planner workflow dependencies across modules

🟠 **Medium-Risk Areas**

- UI rendering issues (forms and view module)
- Activity–Program relationship consistency

🟢 **Low-Risk Areas**

- Static UI components

🧠 **Step 4: Test Strategy Design**

✅ **1. Test Strategy**

🎯 **Objective**

Ensure:

- Functional correctness across all modules
- Secure and accurate role-based access control
- Reliable end-to-end workflow execution
- Stable system performance under expected load

📦 Scope

✓ In Scope

- Login authentication and session management
- RBAC validation across all roles
- Form creation, modification, and deletion
- Organizational hierarchy configuration
- Program and activity management
- Planning workflows (activity assignment and scheduling)
- Data visualization in the View module

❌ Out of Scope

- Third-party integrations (if not directly testable)
- Infrastructure-level failures (network, hardware, etc.)

🧪 Test Approach

🔹 Functional Testing

- Validate all module-level workflows
- Perform CRUD operations validation

🔹 End-to-End Testing

- Validate complete workflow:
  Form → Activity → Planner → Execution → View

🔹 Security Testing

- Validate RBAC restrictions across roles
- Verify unauthorized access scenarios

🔹 Performance Testing

- Validate system behavior under load
- Test large form submissions and planner operations

🔹 Regression Testing

- Execute automated regression suite in CI/CD pipeline

⚙️ CI/CD Strategy

Stage	Testing Type
Build	Smoke Testing
QA	Functional + Regression
Staging	End-to-End + Performance

🧪 Test Data Strategy

- Role-based user datasets (Root/Admin/Basic/Medium)
- Tenant-specific datasets
- Dynamic form input variations

🌐 Environment Strategy

Dev → QA → Staging → Production

✅ Entry Criteria

- Stable build deployed
- Required test data available
- Test environment ready

✅ Exit Criteria

- No critical or high-priority defects
- ≥ 95% test pass rate
- All high-risk scenarios validated

📊 Metrics

- Test coverage (%)
- Defect density
- Test execution rate

- Defect leakage

🧪 2. Test Plan (Module-Based)

📌 Features to be Tested

- Login
- MiPrevsis / Dashboard
- Master Forms
- Organizational Structure
- Master Services
- Planner
- View Module
- User Profiling & Management

🧪 Key Test Scenarios

🔹 Functional Testing

- Login validation (valid/invalid cases)
- Form creation with all field types
- Role-based access validation

🔹 Workflow Testing (Critical)

- Create Form → Assign to Activity → Plan → Execute → Validate in View

---

🔹 Security Testing

- Role restriction validation
  (e.g., Basic user accessing Admin features)

🔹 Performance Testing

- Handling large form configurations
- Multiple planner entries under load

📅 Test Schedule

- Daily Smoke Testing
- Weekly Full Regression

📦 Deliverables

- Test cases
- Defect reports
- Automation execution reports

🤖 Task-2: Automation Framework (Chain-of-Thought Approach)

🧠 Step-by-Step Decisions

Decision Area	Choice	Justification
Framework Type	Hybrid	Scalable & industry standard
Language	TypeScript	Strong typing & maintainability
Tool	Playwright	Modern, fast, reliable
Execution	Hybrid (Local + CI/CD + Cloud)	Flexible & scalable
Design Pattern	POM + BDD + Data-Driven	Best practice
Reporting	Allure	Rich reporting

📐 Framework Structure

```
playwright-framework/
│
├── config/
│   ├── env.config.ts
│   ├── playwright.config.ts
│
├── test-data/
│   ├── login.data.json
```

```
│   ├── user.data.json
│
├── src/
│   ├── pages/
│   ├── locators/
│   ├── utils/
│   ├── hooks/
│   ├── step-definitions/
│   ├── features/
│
├── reports/
│   ├── allure-results/
│   ├── html-report/
│
├── tests/
│   ├── runners/
│
├── ci/
│   ├── github-actions.yml
│   ├── jenkinsfile
│
├── package.json
├── tsconfig.json
└── README.md
```

🔑 Key Components

POM Layer

- Handles UI interactions and page structure

BDD Layer

- Enables business-readable test scenarios

Data Layer

- JSON-based test data management

CI/CD Integration

- Automated execution via GitHub Actions / Jenkins

🔍 Final Review (STRICT – Enterprise Level)

❌ Identified Gaps

1. Limited Validation Depth

- No API-level validation
- No backend data verification

---

2. Weak Negative Testing Coverage

- Missing invalid/corrupt form scenarios
- No boundary or edge-case testing

---

3. Incomplete RBAC Coverage

- No privilege escalation scenarios
- No cross-role validation

---

4. Performance Testing Gaps

- No stress testing
- No spike/load variation scenarios

---

5. Workflow Complexity Under-tested

- Planner dependencies not deeply validated

❌ Automation Gaps

- Missing API automation layer
- No visual testing strategy
- No accessibility testing

- No tagging strategy for test execution
- Parallel execution configuration not detailed

✅ Strengths

- Strong understanding of application modules
- Good workflow coverage
- Correct framework selection
- CI/CD-ready approach

📜 Final Verdict

Area	Score
Test Strategy	7/10
Test Plan	7/10
Automation Framework	8/10

🧠 Final Comment

You demonstrate:

- Good conceptual understanding
- Correct architectural direction

However:

- Depth and edge-case coverage are lacking

👉 In real-world projects:

- This would pass an initial review
- But may fail under production-level validation